Guía de la Exposición

Automatización de las Pruebas y Pipelines

Proyecto: Galletas Natys

Equipo 8 · Sección IN3103

13 de noviembre de 2026

Elaborada por: **Elías Armas**

Índice

1. Cómo usar esta guía
2. Organización de la exposición
3. Parte 1 · Introducción y Fundamentos de la Automatización (Elías Armas)
4. Parte 2 · Pipelines y CI/CD (Moisés Pereira)
5. Parte 3 · JMeter: fundamentos, arquitectura y diseño de scripts (Elías Armas)
6. Parte 4 · Demo en vivo con JMeter (Ismael Torres)
7. Parte 5 · Buenas prácticas, estrategias CI/CD y cierre (Santiago Loyo)
8. Caso práctico: el sistema Natys
9. Preguntas frecuentes del público y cómo responderlas
10. Glosario
11. Plan B: qué hacer si algo falla
12. Lista de verificación antes, durante y después
13. Consejos para hablar en público

1. Cómo usar esta guía

Esta guía reúne en un solo documento todo lo necesario para presentar la exposición **«Automatización de las Pruebas y Pipelines»** del Equipo 8. No es un guion para leer en voz alta: es un material de apoyo para estudiar, ensayar y resolver dudas antes y durante la presentación.

Cada parte de la exposición tiene la misma estructura:

- **Objetivo:** lo que el público debe entender al terminar esa parte.
- **Contenido:** la explicación completa del tema, más amplia de lo que se dirá en voz alta, para que el expositor domine el tema y pueda responder preguntas.
- **Qué decir:** frases sugeridas, marcadas con una línea roja a la izquierda.
- **Tiempo:** duración recomendada de cada bloque.
- **Transición:** la frase para pasar la palabra al siguiente compañero.

Recomendación: leer la guía completa al menos una vez, aunque solo se presente una parte. Si un compañero falta o se queda en blanco, cualquiera debe poder continuar.

Las secciones finales (preguntas frecuentes, glosario, plan B y listas de verificación) sirven para todo el equipo y conviene repasarlas el día anterior a la exposición.

2. Organización de la exposición

2.1 Datos generales

Elemento	Detalle
Tema	Automatización de las Pruebas y Pipelines
Proyecto de referencia	Sistema web de Galletas Natys
Equipo	Equipo 8 · Sección IN3103
Fecha	13 de noviembre de 2026
Duración total sugerida	35 a 40 minutos, más 5 a 10 minutos de preguntas
Material	Diapositivas, tríptico, mapa mental, mapa conceptual, dashboards y código QR de evaluación

2.2 Orden y responsables

#	Expositor	Tema	Tiempo
1	Elías Armas	Introducción y Fundamentos de la Automatización	6–7 min
2	Moisés Pereira	Pipelines y CI/CD	7 min
3	Elías Armas	JMeter: fundamentos, arquitectura y diseño de scripts	7 min

4	Ismael Torres	Demo en vivo con JMeter	8 min
5	Santiago Loyo	Buenas prácticas, estrategias CI/CD y cierre	6 min

2.3 Hilo conductor

La exposición cuenta una historia en cinco pasos: primero explicamos **qué es automatizar pruebas y por qué conviene**; luego mostramos **cómo se ejecutan esas pruebas solas en cada cambio** mediante pipelines; después enseñamos **cómo se diseña una prueba** con JMeter; a continuación lo **demostramos en vivo** sobre el sistema Natys; y cerramos con **buenas prácticas** para aplicarlo en cualquier proyecto.

Idea central que todo el público debe recordar: **«Automatiza lo repetitivo y deja a las personas lo que requiere criterio».**

2.4 Roles durante la exposición

- **Quien expone:** habla, señala la diapositiva y mantiene el contacto visual.
- **Quien controla las diapositivas:** un compañero que no está exponiendo avanza las diapositivas cuando el expositor lo indica.
- **Quien controla el tiempo:** hace una seña discreta al faltar 1 minuto.
- **Soporte técnico:** Ismael prepara la demo y los videos de respaldo; los demás no tocan su computadora durante la presentación.

3. Parte 1 · Introducción y Fundamentos de la Automatización

Expositor: **Elías Armas** · Tiempo: 6–7 minutos

Objetivo

Que el público entienda qué es la automatización de pruebas, en qué se diferencia de las pruebas manuales, cuándo conviene aplicarla y cómo se distribuyen las pruebas en la pirámide.

3.1 Rompehielo (1 minuto)

Abrir con una pregunta que involucre al público. Funciona mejor si se pide levantar la mano, porque rompe la tensión y hace que la audiencia preste atención desde el inicio.

> *«Buenos días. Antes de empezar, una pregunta: ¿quién ha tenido que probar lo mismo una y otra vez en su proyecto, por ejemplo iniciar sesión diez veces para ver si ya funciona? Levanten la mano… Exactamente. Hoy les vamos a mostrar cómo dejar que la computadora haga ese trabajo por nosotros.»*

Alternativa: preguntar «¿Cuánto creen que tardaría una persona en probar 100 compras seguidas en una tienda en línea?» y luego revelar que una herramienta lo hace en segundos.

3.2 ¿Qué es la automatización de pruebas? (1 minuto)

La automatización de pruebas consiste en usar **scripts y herramientas** que ejecutan las pruebas de un sistema de forma automática, comparan el resultado obtenido con el resultado esperado y generan un reporte, sin que una persona tenga que hacer cada paso a mano.

Una prueba automatizada tiene tres elementos:

- **Datos de entrada:** por ejemplo, un usuario y una contraseña.
- **Acción:** lo que se ejecuta, como enviar el formulario de inicio de sesión.
- **Verificación:** la comprobación de que el resultado es el esperado, como que aparezca el panel principal.

Si la verificación se cumple, la prueba **pasa**; si no, **falla** y avisa del error.

> *«Automatizar una prueba es escribirla una vez y poder ejecutarla mil veces, siempre igual y sin cansarse.»*

3.3 Pruebas manuales vs. automáticas (1–2 minutos)

Criterio	Prueba manual	Prueba automatizada
Velocidad	Lenta: depende de la persona	Rápida: minutos o segundos
Repetición	Cansada y propensa a errores	Idéntica cada vez
Costo inicial	Bajo	Alto: hay que crear los scripts
Costo a largo plazo	Alto: se paga cada ejecución	Bajo: la ejecución es casi gratis
Criterio humano	Sí: detecta problemas visuales o de experiencia	No: solo verifica lo programado
Ideal para	Exploración, usabilidad, casos únicos	Regresión, carga, flujos críticos

Ninguna reemplaza por completo a la otra: se complementan. La automatización libera tiempo para que las personas hagan las pruebas que requieren criterio.

3.4 Cuándo sí y cuándo no automatizar (1 minuto)

Sí conviene automatizar

- **Pruebas de regresión:** verificar que lo que ya funcionaba sigue funcionando después de cada cambio.
- **Pruebas de carga y rendimiento:** simular muchos usuarios a la vez, algo imposible de hacer a mano.
- **Flujos críticos del negocio:** en Natys, el inicio de sesión, el carrito y los pedidos.
- **Pruebas con muchos datos:** probar el mismo formulario con cientos de combinaciones.

No conviene automatizar

- Pruebas que se harán **una sola vez**.
- Pantallas que **cambian a diario**: el script se rompería constantemente.
- Pruebas de **usabilidad** o de apariencia, que dependen de la opinión humana.

3.5 ROI y pirámide de pruebas (1–2 minutos)

Retorno de la inversión (ROI)

Automatizar cuesta tiempo al principio. La inversión se recupera cuando la prueba se repite muchas veces. Ejemplo: si probar el carrito a mano toma 10 minutos y se hace 3 veces por semana, en un semestre son unas 12 horas. Escribir el script puede tomar 2 horas y luego cada ejecución dura segundos: el ahorro es evidente.

Fórmula sencilla: **ROI = (ahorro obtenido − costo de automatizar) / costo de automatizar**.

Pirámide de pruebas

Nivel	Cantidad	Velocidad	Ejemplo en Natys
UI / extremo a extremo (cima)	Pocas	Lentas y frágiles	Comprar galletas desde el navegador
Integración / API (medio)	Moderadas	Intermedias	El carrito se comunica con la base de datos
Unitarias (base)	Muchas	Muy rápidas	La función que calcula el total del pedido

> *«La pirámide nos dice que pongamos muchas pruebas pequeñas y rápidas abajo, y pocas pruebas grandes arriba. Así detectamos los errores pronto y barato.»*

Transición

> *«Ya sabemos qué automatizar y por qué. Ahora Moisés nos va a mostrar cómo lograr que estas pruebas se ejecuten solas cada vez que alguien cambia el código: los Pipelines y CI/CD.»*

4. Parte 2 · Pipelines y CI/CD

Expositor: **Moisés Pereira** · Tiempo: 7 minutos

Objetivo

Que el público entienda qué es un pipeline, la diferencia entre Integración, Entrega y Despliegue Continuo, y cómo las pruebas automatizadas detienen un despliegue defectuoso.

4.1 ¿Qué es un pipeline?

Un pipeline es una **serie de etapas automáticas** por las que pasa el código desde que el programador lo guarda hasta que llega a los usuarios. Cada etapa debe terminar bien para pasar a la siguiente; si una falla, el proceso se detiene.

Etapa	Qué ocurre
1. Commit	El programador sube su cambio al repositorio (por ejemplo, GitHub).
2. Build	Se instalan dependencias y se construye la aplicación.
3. Pruebas	Se ejecutan las pruebas automatizadas: unitarias, de integración y otras.
4. Despliegue	Si todo pasó, la aplicación se publica en un servidor de pruebas o de producción.

> *«Un pipeline es como una línea de producción de galletas: cada galleta pasa por control de calidad, y si una sale mal, no llega a la caja.»*

4.2 Integración, Entrega y Despliegue Continuo

- **Integración Continua (CI):** cada cambio se integra al repositorio principal varias veces al día y se verifica automáticamente con build y pruebas.
- **Entrega Continua (CD – Delivery):** además de verificar, el código queda siempre listo para publicarse; la publicación final la aprueba una persona.
- **Despliegue Continuo (CD – Deployment):** si todas las pruebas pasan, el cambio se publica automáticamente sin intervención humana.

Diferencia clave: en la **Entrega** Continua una persona presiona el botón de publicar; en el **Despliegue** Continuo lo hace la máquina.

4.3 Disparadores (triggers)

Un pipeline no se ejecuta solo porque sí: lo inicia un evento, llamado disparador.

- **Commit / push:** cada vez que se sube código.
- **Pull request:** cuando alguien pide integrar su rama a la principal.
- **Horario:** por ejemplo, todas las noches a las 2:00 a. m. (pruebas largas o de carga).
- **Manual:** un botón para ejecutarlo cuando se necesite.

4.4 Ejemplo con GitHub Actions y PHPUnit

GitHub Actions permite definir un pipeline en un archivo YAML dentro del repositorio. Ejemplo simplificado:

```
name: Pruebas Natys
on: [push, pull_request]
jobs:
  pruebas:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: composer install
      - run: vendor/bin/phpunit
```

Lectura: «cada vez que haya un push o un pull request, en una máquina Ubuntu, descarga el código, instala dependencias y ejecuta PHPUnit». Si una prueba falla, GitHub marca el cambio en rojo y el despliegue no ocurre.

> *«Si una sola prueba falla, el pipeline se detiene. El error nunca llega al cliente.»*

4.5 Beneficios

- Los errores se detectan **minutos** después de escribirse, no semanas después.
- Corregir un error pronto es mucho más barato que corregirlo en producción.
- El equipo publica con más confianza y con más frecuencia.
- Queda un historial de qué cambio rompió qué prueba.

Transición

«El pipeline ejecuta pruebas, pero ¿cómo se escribe una buena prueba? Elías nos lo explica con JMeter.»

5. Parte 3 · JMeter: fundamentos, arquitectura y diseño de scripts

Expositor: **Elías Armas** · Tiempo: 7 minutos

Objetivo

Que el público sepa qué es un script de prueba, cómo se estructura, qué principios y patrones se usan para diseñarlo y cuáles son los componentes de un plan de pruebas en JMeter.

5.1 El script de prueba y la estructura AAA

Un script de prueba es un conjunto de instrucciones que ejecuta una prueba automáticamente. La forma más clara de organizarlo es el patrón **AAA**:

Fase	Significado	Ejemplo: agregar al carrito
Arrange (Preparar)	Dejar todo listo: datos, usuario, estado inicial	Iniciar sesión y tener el carrito vacío
Act (Actuar)	Ejecutar la acción que se quiere probar	Agregar 2 cajas de galletas
Assert (Verificar)	Comprobar el resultado	El carrito muestra 2 productos y el total correcto

5.2 Principios de un buen script

- **Modular:** dividido en partes reutilizables (por ejemplo, un bloque de «iniciar sesión» que usan varias pruebas).
- **Independiente:** cada prueba funciona sola, sin depender del resultado de otra.
- **Legible:** cualquiera del equipo puede entender qué hace.
- **Nombres claros:** «Login con contraseña incorrecta muestra error» es mejor que «prueba7».
- **Repetible:** da el mismo resultado cada vez que se ejecuta en las mismas condiciones.

5.3 Patrones de diseño

- **Guiado por datos (data-driven):** la misma prueba se ejecuta con muchos datos leídos de un archivo CSV. Ejemplo: 100 usuarios distintos iniciando sesión.
- **Guiado por palabras clave (keyword-driven):** las acciones se describen con palabras como «Abrir», «Escribir», «Hacer clic», que incluso personas no programadoras pueden combinar.
- **Page Object:** cada pantalla del sistema se representa como un objeto con sus acciones; si la pantalla cambia, solo se corrige en un lugar.
- **Por capas:** se separan las pruebas, la lógica de negocio y el acceso a la interfaz o API.

5.4 ¿Qué es JMeter?

Apache JMeter es una herramienta **de código abierto**, escrita en **Java**, diseñada para pruebas de **carga y rendimiento**. Permite simular cientos o miles de usuarios usando un sistema al mismo tiempo y medir cómo responde.

- **Gratuita** y con una comunidad grande.
- **Multiplataforma:** funciona en Windows, Linux y macOS.
- **Multiprotocolo:** HTTP/HTTPS, bases de datos (JDBC), FTP, servicios REST y SOAP, entre otros.
- **Multihilo:** cada usuario virtual es un hilo que trabaja en paralelo.
- **Integrable** en pipelines de CI/CD desde la línea de comandos.

5.5 Componentes de un plan en JMeter

Componente	Función	Ejemplo Natys
Test Plan	Contenedor de toda la prueba	«Compra de galletas»
Thread Group	Define cuántos usuarios, en cuánto tiempo y cuántas repeticiones	100 usuarios en 10 segundos
Sampler	La petición que se envía	GET /catalogo, POST /carrito
Assertion	Verifica la respuesta	Código 200 y el texto «Pedido registrado»
Listener	Muestra resultados	Tabla de resultados, gráficos, dashboard HTML

Config Element	Configuración compartida	CSV con usuarios, cookies, URL base
Timer	Pausas entre peticiones	Esperar 1–3 s como un usuario real

5.6 Métricas que se leen en JMeter

- **Tiempo de respuesta:** cuánto tarda el servidor en contestar (promedio, mínimo, máximo, percentil 90).
- **Throughput:** cuántas peticiones por segundo soporta el sistema.
- **Porcentaje de error:** cuántas peticiones fallaron.
- **Usuarios concurrentes:** cuántos usuarios estaban activos a la vez.

Transición

> *«Ya vimos la teoría. Ahora Ismael lo va a poner en práctica: 100 usuarios comprando galletas en Natys al mismo tiempo.»*

6. Parte 4 · Demo en vivo con JMeter

Expositor: **Ismael Torres** · Tiempo: 8 minutos

Objetivo

Demostrar en vivo una prueba de carga sobre el sistema Natys y comparar el resultado de una versión con fallas contra una versión que funciona correctamente.

6.1 Preparación previa

- JMeter abierto con el plan ya cargado, antes de que empiece la exposición.
- Servidor de Natys encendido y probado esa misma mañana.
- Archivo CSV de usuarios de prueba en la ruta correcta.
- Los dos videos de respaldo abiertos en otra ventana.
- Zoom de pantalla aumentado para que se lea desde el fondo del salón.

6.2 Flujo probado

Paso	Petición	Verificación
1. Login	POST con usuario y contraseña del CSV	Respuesta 200 y sesión iniciada
2. Catálogo	GET de la lista de productos	Aparecen los productos
3. Carrito	POST para agregar galletas	El carrito tiene el producto
4. Pago	POST para confirmar el pedido	Mensaje de pedido registrado

6.3 Guion de la demo

1. Mostrar el Test Plan y explicar en 30 segundos cada componente que se ve.
2. Mostrar el Thread Group: «Vamos a simular 100 usuarios en 10 segundos».
3. Ejecutar la **versión con fallas** y mostrar en el dashboard los errores en rojo y los tiempos altos.
4. Ejecutar la **versión corregida** y mostrar el dashboard con 0 % de error.
5. Comparar ambos resultados lado a lado.
6. Mostrar el código QR con los dashboards y el formulario.

> *«Así se ve un sistema que no aguanta la carga… y así se ve después de corregirlo y verificarlo con pruebas automáticas.»*

6.4 Qué señalar en el dashboard

- Gráfico de tiempo de respuesta: en la versión con fallas sube mucho.
- Tabla de errores: qué peticiones fallaron y por qué.
- APDEX: índice de satisfacción del usuario (1 es perfecto).
- Throughput: cuántas compras por segundo soporta el sistema.

6.5 Plan de respaldo

Si la demo falla (sin internet, servidor caído o error inesperado), **no improvisar**: decir con calma «Para no perder tiempo, les muestro la grabación de esta misma prueba» y reproducir los dos videos.

Transición

> *«Ya lo vieron funcionando. Para cerrar, Santiago nos dará las buenas prácticas para aplicar todo esto en cualquier proyecto.»*

7. Parte 5 · Buenas prácticas, estrategias CI/CD y cierre

Expositor: **Santiago Loyo** · Tiempo: 6 minutos

Objetivo

Dar al público un conjunto de recomendaciones prácticas y cerrar la exposición con un mensaje claro e invitar a la evaluación.

7.1 Ecosistema de herramientas

Herramienta	Uso principal
JMeter	Pruebas de carga y rendimiento
PHPUnit	Pruebas unitarias en PHP
Selenium	Pruebas de interfaz en el navegador
Postman	Pruebas de APIs
Jenkins	Servidor de integración continua
GitHub Actions	Pipelines integrados en GitHub

7.2 Buenas prácticas

- **Shift-left:** probar lo antes posible en el ciclo de desarrollo, no al final.
- **Paralelizar pruebas:** ejecutar varias a la vez para que el pipeline sea rápido.
- **Entornos aislados:** probar en un ambiente separado de producción, con datos de prueba.
- **Puertas de calidad (quality gates):** reglas que impiden publicar si, por ejemplo, la cobertura baja de 80 % o hay pruebas fallando.
- **Mantener las pruebas:** una prueba desactualizada es peor que ninguna, porque da falsa confianza.
- **Pruebas rápidas primero:** unitarias al inicio del pipeline y las largas al final o por la noche.

7.3 Beneficio para el negocio

Con pruebas automatizadas y pipelines, el **time-to-market** (tiempo para llevar una mejora al cliente) se reduce, porque el equipo publica con confianza sin largas rondas de pruebas manuales.

7.4 Cierre

> *«Para resumir: automatiza lo repetitivo y deja a las personas lo que requiere criterio. Gracias por su atención. Les pedimos que escaneen el código QR y respondan la evaluación; nos ayuda mucho.»*

Después del cierre, abrir el espacio de preguntas. Todos los integrantes pasan al frente.

8. Caso práctico: el sistema Natys

Toda la exposición usa como ejemplo el sistema web de **Galletas Natys**, una aplicación para gestionar la producción y venta de galletas. Conocerlo ayuda a dar ejemplos concretos y a responder preguntas.

8.1 Módulos del sistema

Área	Módulos
Ventas	Pedidos, Ecommerce, Carrito de compras, Clientes, Tipos de clientes, Promociones
Producción	Productos, Producciones, Materias primas, Unidades de medida, Categorías
Compras y finanzas	Compras, Proveedores, Pagos, Cuentas por cobrar, Cuentas por pagar, Moneda, Tasa cambiaria
Logística	Entregas, Camiones
Administración	Usuarios, Roles y permisos, Módulos, Bitácora, Respaldos, Notificaciones, Dashboard
Atención	Chatbot

8.2 Informes de pruebas realizados

El equipo elaboró informes de pruebas por módulo, publicados en la página web de la exposición, en la sección «Pruebas». Entre ellos: Pedidos, Carrito de compras, Moneda, Módulos, Roles, Productos, Compras, Producciones, Entregas y Chatbot. Todos se pueden descargar juntos en un solo PDF.

8.3 Por qué Natys es un buen ejemplo

- Tiene flujos críticos claros (compra y pedido) ideales para automatizar.
- Puede recibir muchos clientes a la vez en temporadas altas: justifica pruebas de carga.

- Tiene muchos módulos que cambian: justifica pruebas de regresión en un pipeline.

9. Preguntas frecuentes del público

Preguntas que probablemente haga el profesor o el público, con una respuesta corta sugerida. Responde quien domine el tema; si nadie la sabe, decir: «Buena pregunta, la investigamos y se la respondemos».

¿La automatización reemplaza a los testers?

No. Cambia su trabajo: el tester deja de repetir pruebas y se dedica a diseñarlas, analizar resultados y hacer pruebas exploratorias que requieren criterio.

¿Cuánto cuesta automatizar?

Las herramientas que mostramos (JMeter, PHPUnit, Selenium, GitHub Actions en repositorios públicos) son gratuitas. El costo principal es el tiempo de crear y mantener los scripts.

¿Qué diferencia hay entre prueba de carga y prueba de estrés?

La de carga verifica el comportamiento con la cantidad de usuarios esperada; la de estrés supera ese límite para encontrar el punto en que el sistema falla.

¿Por qué JMeter y no otra herramienta?

Es gratuita, madura, multiprotocolo, con mucha documentación, y se integra fácil con pipelines. Alternativas: k6, Gatling, Locust.

¿Qué pasa si una prueba falla en el pipeline?

El pipeline se detiene, el cambio se marca en rojo y no se despliega. El programador revisa el reporte, corrige y vuelve a subir el cambio.

¿Qué es mejor, Jenkins o GitHub Actions?

Depende. Jenkins se instala en servidor propio y es muy configurable; GitHub Actions viene integrado con GitHub y es más fácil de empezar. Para proyectos pequeños, GitHub Actions es más práctico.

¿Cuántas pruebas hay que automatizar?

No existe un número fijo. Se empieza por los flujos críticos y las pruebas que más se repiten, siguiendo la pirámide: muchas unitarias, menos de integración y pocas de interfaz.

¿Qué es una prueba «flaky»?

Una prueba inestable que a veces pasa y a veces falla sin cambios en el código, normalmente por depender del tiempo, la red o el orden. Hay que corregirla o aislarla porque resta confianza.

¿Cómo se prueba algo que necesita base de datos?

Con una base de datos de prueba aislada, que se llena con datos conocidos antes de cada prueba y se limpia después.

¿Qué significa cobertura de código?

El porcentaje del código que se ejecuta durante las pruebas. Una cobertura alta ayuda, pero no garantiza que las pruebas verifiquen lo correcto.

10. Glosario

Término	Definición
AAA	Arrange, Act, Assert: preparar, actuar y verificar.
APDEX	Índice de satisfacción del usuario según los tiempos de respuesta (0 a 1).
Assertion	Verificación de que una respuesta cumple lo esperado.
Build	Proceso de construir la aplicación a partir del código fuente.
CI	Integración Continua: integrar y verificar cambios con frecuencia.
CD	Entrega o Despliegue Continuo.
Commit	Registro de un cambio en el repositorio.
CSV	Archivo de texto con datos separados por comas, usado para pruebas guiadas por datos.
Dashboard	Panel visual con los resultados de las pruebas.
Flaky test	Prueba inestable que pasa o falla sin cambios en el código.
Listener	Componente de JMeter que muestra resultados.
Page Object	Patrón que representa cada pantalla como un objeto reutilizable.
Percentil 90	Tiempo por debajo del cual está el 90 % de las respuestas.
Pipeline	Serie de etapas automáticas desde el commit hasta el despliegue.

Pull request	Solicitud para integrar cambios de una rama a otra.
Quality gate	Regla que bloquea la publicación si no se cumplen criterios de calidad.
Regresión	Error en algo que antes funcionaba, causado por un cambio nuevo.
ROI	Retorno de la inversión.
Sampler	Componente de JMeter que envía una petición.
Shift-left	Mover las pruebas al inicio del ciclo de desarrollo.
Thread Group	Grupo de usuarios virtuales en JMeter.
Throughput	Cantidad de peticiones atendidas por segundo.
Time-to-market	Tiempo para llevar una mejora al cliente.
Trigger	Evento que inicia un pipeline.
YAML	Formato de texto usado para configurar pipelines.

11. Plan B: qué hacer si algo falla

Problema	Solución
No funciona el proyector	Usar el tríptico impreso y explicar con el mapa mental; mostrar la laptop a las primeras filas.
No hay internet	Las diapositivas y los videos deben estar descargados en la computadora (no solo en línea).
Falla la demo de JMeter	Reproducir los videos de respaldo (prueba que falla y prueba que funciona).
Falta un compañero	Su parte la toma quien presenta antes o después; todos estudiaron la guía completa.
Alguien se queda en blanco	El compañero más cercano hace una pregunta guía o lee el siguiente punto de la diapositiva.
Se acaba el tiempo	Saltar los ejemplos y ir directo a las frases clave y la transición.
Pregunta difícil	«Buena pregunta; la investigamos y se la respondemos». Nunca inventar.
No carga el código QR	Dictar o escribir en la pizarra la dirección corta del formulario.

12. Lista de verificación

Un día antes

- ☐ Ensayo completo cronometrado con todo el equipo.
- ☐ Diapositivas finales descargadas en PPTX y PDF en al menos dos computadoras y un pendrive.
- ☐ Videos de respaldo de la demo descargados.
- ☐ Tríptico impreso.
- ☐ Código QR probado desde varios teléfonos.
- ☐ Página web de la exposición revisada.
- ☐ Acordar vestimenta del equipo.

El día de la exposición

- ☐ Llegar 20 minutos antes.
- ☐ Probar proyector, cable HDMI, sonido y control de diapositivas.
- ☐ Abrir JMeter, el plan de pruebas y los videos.
- ☐ Encender el servidor de Natys y hacer una prueba rápida.
- ☐ Silenciar teléfonos y notificaciones de la computadora.
- ☐ Botella de agua para los expositores.

Después

- ☐ Recordar al público llenar la evaluación.
- ☐ Agradecer al profesor.
- ☐ Anotar las preguntas que no se pudieron responder.

13. Consejos para hablar en público

Antes de hablar

- Respirar profundo tres veces antes de pasar al frente.
- Conocer muy bien la primera frase: el inicio es lo que más nervios da.
- Ensayar en voz alta, no solo leyendo en silencio.

Mientras se habla

- **Mirar al público,** no a la diapositiva. La diapositiva es apoyo, no un guion.
- **Hablar despacio** y hacer pausas después de las ideas importantes.
- **Voz alta y clara,** pensando en la última fila.
- **Postura abierta:** no cruzar los brazos ni dar la espalda.
- **Usar ejemplos de Natys:** lo concreto se entiende y se recuerda mejor.
- **No leer párrafos completos:** usar palabras clave como recordatorio.

Si algo sale mal

- Mantener la calma: el público casi nunca nota los errores pequeños.
- Si se olvida algo, mirar la diapositiva, respirar y seguir con el siguiente punto.
- Sonreír: transmite seguridad.

Mensaje final del equipo: «Automatiza lo repetitivo y deja a las personas lo que requiere criterio». ¡Éxito, Equipo 8!